

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

1 WHY ADVANCED PYTHON?

★ What is Advanced Python?

- Beyond basics OOP functions, mastering internals, decorators, generators, context managers, metaclasses, GIL.

Advanced = Decorators +
Generators + Context Managers
+ Metaclasses
+ Internals

★ What is Why Advanced?

- Basics gets junior job, advanced gets senior Staff FAANG, interview asks advanced internals.

★ What is Python Execution Model?

- Code → Bytecode .pyc → Python Virtual Machine PVM interprets.

★ What is import this?

- Zen of Python
- Beautiful is better than ugly, Explicit is better than implicit, Simple is better than complex.

BASIC	vs	ADVANCED
Basic =	syntax loops functions dicts	functions OOP lists junior level
Advanced =	decorators generators context metaclasses	GIL memory senior Staff FAANG

Roadmap of 20 slides:

- Decorators, Closures, Generators, Iterators, Context Managers, Metaclasses, GIL, Memory Management, Magic Methods, Descriptors, etc.
- + Inheritance vs Composition, Async/Await, Concurrency, Threading, Properties, Descriptors Protocol, Weakrefs, __slots__, Import system, C-extensions, Performance profiling, Typing/TypeHints, Dataclasses, LRU Cache

Key Terms

Advanced Python
- beyond basics senior Staff
FAANG • Execution Model
- code → bytecode → PVM
• Bytecode
- .pyc compiled

PVM - Python Virtual Machine
interprets
• Zen of Python
• Beautiful
• Explicit
• Simple

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

2

SECTION 2: CLOSURES & FIRST-CLASS FUNCTIONS.

★ What is First-Class Functions?

Definition: Functions treated as objects — can be passed as args, returned, assigned to variable. Python functions are first-class.

- Examples:
- `def greet(name): return f"Hello {name}"`
- `assign greet_func = greet`
`greet_func("John")` → Hello John
- pass as arg:
- `def call(func): return func("John")`
- `call(greet)` → Hello John

★ What is Closure?

Definition: Function remembers variables from enclosing scope even after outer function finished.

Example:

- `def outer(x):`
 `def inner(y): return x + y`
 `return inner`
- `closure = outer(10)`
 `closure(5) = 15` remembers `x=10`
- Why Closure useful?
 - Data encapsulation → private variables
 - decorator factory — interview favorite

FIRST-CLASS	CLOSURE
First-Class = functions passed / returned / assigned	as objects / resigned

★ What is Nested Functions?

Function inside function

- `def outer():`
- `def inner(): print("inner")`
 `inner()`
 `outer()` → prints: inner

★ What is nonlocal?

Modify enclosing variable — not global

- `def outer(): x = 10`
- `def inner():`
 `nonlocal x`
 `x = 20`
 `inner()`
 `print(x)` → 20

Example Closure Counter

- `def counter():`
- `count = 0`
- `def increment():`
 `nonlocal count`
 `count += 1`
 `return count`
 `return increment`
- `c = counter()`
 `c()` → 1
 `c()` → 2 remembers

Key Terms: Definitions -

- First-Class: functions as objects, passed / returned / assigned to variable
- Closure: remembers enclosing scope after outer finished → data encapsulation / private
- Nested function: function defined inside another function
- nonlocal: modify enclosing scope variable, not global
- Closure factory: counter remembers count — common interview pattern

PYTHON ADVANCED HANDBOOK

Page 3/20

50 Must-Know Concepts

3 SECTION 3: DECORATORS - MOST IMPORTANT TRAP

★ What is Decorator?

Definition: Decorator = Function that takes another function adds functionality wraps without modifying original code
@decorator syntax syntactic sugar.

How Decorator Works?

```
def my_decorator(func):  
    : def wrapper(  
    : print "Before"  
    : func(  
    : print "After"  
    : return wrapper  
    @my_decorator.  
    : print "Hello) # Before Hello After
```

Decorator with Arguments

```
def decorator(func):  
    : def wrapper(*args, **karins):  
    : print "Before"  
    : result = func(*args, **karins):  
    : print "After"  
    : return result  
    @decorator  
    def add(a, b): return a + b  
    add(2, 3) → Before After 5
```

Example: Decorator Factory

```
def repeat(n:: func):  
    : def wrapper(*args, **karins):  
    : for _ in range(n):  
    : func(*args, **karins):  
    : return wrapper  
    : return decorator  
def hello:  
    : print "Hello"  
    : hello( → prints Hello 3 times.
```

Real World Decorators:

- @property – getter
- @staticmethod
- @classmethod
- @lru_cache – caches
- @login_required

DECORATOR TYPES

Type	Example	Note
Simple	@decorator	no args wrapper no args
With	*args/*karins)	recommended
Factory	@repeat(n)	with args – factory returns decorator

Key Terms: Definitions –

- Decorator: function that takes function adds wraps – @syntax = syntactic sugar
- wrapper: inner function that adds before/after logic
*args/*karins: handles any args – recommended
- Factory: decorator with args, e.g. @repeat(n)
- Real decorators: @property, @staticmethod, @classmethod, @lru_cache, @login_required

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

4

SECTION NAME:

4 DECORATORS ADVANCED

★ What is functools.wraps? :

Preserves original function metadata → name, docstring. Without wraps wrapper name becomes "wrapper" loses original metadata. With wraps keeps original. Then

- `functools.wraps` preserves `__name__`, `__doc__`, `__module__`

Example: functools.wraps usage

```
import functools
def decorator(func):
    @functools.wraps(func) # keep metadata
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
@decorator
def hello_doc():
    print("Hello")
hello.__name__ # with wraps → "hello"
# without wraps → "wrapper"
```

★ What is Class Decorator? :

Decorator applied to a class → adds functionality/attrs to the class. Common for auto-registration, injecting methods.

- Class decorator adds method/attr to class

Example: Class Decorator

```
def add_method(cls):
    cls.new_method = lambda self: "new" # added fn
    return cls
@add_method
class MyClass:
    pass
# @decorator1 @decorator2 → decorator1(decorator2(func))
```

★ What is Practical Decorators? :

Function decorator for reusable behavior like timing, logging, caching. Use time to measure execution time.

Example: Timing decorator

```
import time
def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end-start:.4f}s")
        return result
    return wrapper
```

```
@timer
def slow():
    time.sleep(1)
    return 'done'
slow() # → prints "slow took 1.0012s"
```

STACKING ORDER TABLE: | Order | Execution

@decorator1	@decorator2	def func()
→ decorator2	wraps func	FIRST (bottom)
→ then decorator1	wraps decorator2 result	(top)
TRAP! Order matters → decorators run top→bottom in definition, but execute bottom→top		

★ What is Decorator Use Cases? :

- Logging → `@log` logs args/return |
- Timing → `@timer` measures runtime |
- Caching → `@lru_cache` / `@cache` store results |
- Auth → `@login_required` check user |
- Validation → `@validate` validate inputs

★ Key Terms or Interview Tips:

- `functools.wraps` preserves metadata (name, docstring, module) → wrapper keeps original |
- Class Decorator → adds methods/attrs to class via `@add_method` |
- Stacking → multiple decorators apply bottom-up: decorator2 wraps func first, then decorator1 wraps that |
- Timing decorator → use `time.time()` to measure duration |
- Common uses: Logging `@log` | Caching `@lru_cache`/`@cache` | Auth `@login_required`

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

5

SECTION 5 ITERATORS & ITERABLES

★ What is Iterable?

Definition:

- Object that can be iterated over, can return iterator
- `__iter__` method
- list tuple dict set string are iterables
- for loop works

★ What is Iterator?

Definition:

- Object that returns next item
- `__next__` method and `__iter__`
- maintains state, remembers position

How Iterator Works?

- `iter()` gets iterator
- `next()` gets next item
- `StopIteration` when no more

Example code:

```
my_list = [1, 2, 3]
iterator = iter(my_list)
next(iterator) → 1
next(iterator) → 2
next(iterator) → 3
next(iterator) → StopIteration
```

★ How to Create Custom Iterator?

class MyRange:

- `def __init__(self, start, end):`
 - `self.start = start`
 - `self.end = end`
- `def __iter__(self):`
 - `return self`
- `def __next__(self):`
 - `if self.start >= self.end:`
 - `raise StopIteration`
 - `val = self.start`
 - `self.start += 1`
 - `return val`
- `for i in MyRange(0,3):`
 - `print(i) → 0 1 2`

ITERABLE vs ITERATOR

Aspect	Iterable vs Iterator
Iterable	- can be iterated - <code>__iter__</code> returns iterator - list tuple dict
Iterator	- returns next item - <code>__next__</code> & <code>__iter__</code> - maintains state - <code>next()</code>, <code>StopIteration</code>

- for loop works

interview favorite

★ What is Difference?

- Iterable has `__iter__` → returns iterator
- Iterator has `__iter__` and `__next__` → returns next, maintain state

★ What is `iter()` and `next()`?

- `iter(iterable)` → returns iterator
- `next(iterator)` → returns next item

★ Key Terms:

- Iterable - can be iterated; `__iter__` returns iterator; list tuple dict set string; for loop
- Iterator - returns next item; `__next__` & `__iter__`; maintains state position; `next()` `StopIteration`
- `itex()` - gets iterator from iterable
- `next()` - gets next item from iterator
- Custom Iterator - class; `__iter__` returns self; `__next__` returns next or raises `StopIteration`

PYTHON ADVANCED HANDBOOK

LOCKED
EDITION •
UPDATED

50 Must-Know Concepts

Section 6 — GENERATORS — MOST IMPORTANT

What is?

Generator = Special iterator + yield
+ Lazy + Memory efficient

★ What is Generator:

- Special iterator that uses yield instead of return
- Lazy evaluation: generates values on demand
- Memory efficient: doesn't store all values in
- Produces items one at a time when iterated
- Ideal for large datasets / streams / infinite sequences

How Generator Works:

```
def my_gen: 1
• yield 1
• yield 2
• yield 3
gen = my_gen():
• print-(next(gen)): # 1
• print-(next(gen)): # 2
• print-(next(gen)): # 3
• print-(next(gen)): # StopIteration
```

★ Execution flow:

- Call my_gen() → returns generator object, no code runs yet • next() resumes function, runs until yield → pauses & saves state
- resume on next call → continues after yield
- when no yield left → raises StopIteration

★ def vs yield — Core Difference:

def (return)	yield (generator)
Returns a value to caller	Yields a value & pauses function
Function ends immediately	Function state is saved, resumes on next()
Runs fully, then done	Can be resumed multiple times

★ Interview Favorite Trap: return ends; yield pauses

★ Generator Expression vs List Comprehension:

```
gen_expr = (x*x for x in range(10)) # () not [] → generator, lazy, memory efficient
lst = [x*x for x in range(10)] — # [] → list, all in memory at once
```

★ Advantages:

- Memory efficient for large data: don't store all in memory
- Lazy: produce items only when requested
- Can represent infinite sequences via while True yield
- Faster for pipelines: stream processing

Example: Infinite Generator

```
def infinite(n=0):
• while True:
• gen = infinite():
• print-(next(gen)) # 0
• print-(next(gen)) # 1
• print-(next(gen)) # 2
... infinite but lazy, no memory overload
```

Example: Large File Reading

```
def read_large(file_path):
• with open(file_path) as file:
• yield line.strip() # one line at a time:
• for line in read_large("big.txt"):
• process(line) # memory efficient, 1 line processed at a time
```

KEY TERMS — ALL IN ONE:

- Generator: function with yield, returns iterator • yield: pauses + yields value + saves state
- next(): resume generator to next yield • StopIteration: raised Lazy Evaluation: compute on-demand
- Generator Expression: (expr for ...) memory efficient. • Iterator: object with __next__ method
- Iterables: can be looped over • Pipeline: chain generators for memory-efficient processing

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

★ SECTION 7 - GENERATORS ADVANCED

★ What is yield from? —

Delegates to sub-generator.

Syntax `yield from sub-gen()` delegates whole generator. Hands control to sub-generator until it is exhausted. Equivalent to looping and yielding each item.

Example:

Example:

```
; yield 1
; yield 2
; yield from sub() # delegates to sub()
; yield 3
gen = main(). → [1, 2, 3]
note: yield from sub includes sub's yields — outputs 1,2,3
```

★ What is Generator send()??

Sends value INTO generator, generator as coroutine.

Call `.send(value)` to send value into the generator of current yield. Generator pauses at yield and receives sent value as the result of yield expression.

Example:

```
def gen:
; x = yield 1 # x receives sent value
; print-f"Got {x}" # prints Got hello
; yield 2
g = gen(). # 1 # starts, yields 1
g.send('hello') # Got hello → 2
send() resumes generator and passes 'hello' to x
```

Example:

```
g.throw(ValueError) # raises ValueError inside
g.close() # stops gen; further iteration
# Shuts StopIteration generation
Useful for cleaning up resources / early stop
```

★ What is Coroutine? — Generator used as coroutine (async-like before `async/await`).

`throw(exc)`

A generator that yields/awaits values to pause and resume. Supports cooperative multitasking. Precursor to `async/await` in Python.

★ What is itertools?

Module for efficient iterators: infinite & combinational, fast.

Built-in module offering memory-efficient iterators for looping, combinatorics, and infinite streams. No lists created — lazy evaluation.

Must-Know list dense:

- `count()` infinite 0,1,2... → `itertools.count(0)` yields 0,1,2,3,...
- `cycle()` cycles through iterable A B C A B C... → `itertools.cycle("A","B","C")` repeats.
- `repeat()` repeats value `repeat(10,3)` → 10 10 10
- `chain()` chains iterables `chain([1,2],[3,4])` → 1 2 3 4
- `islice()` slices iterator `islice(range(10),2,5)` → 2 3 4
- `product()` cartesian product `product([1,2],("a","b"))` → (1,"a") (1,"b") (2,"a") (2,"b")
- `permutations()` permutations("AB") → AB, BA (order matters)
- `combinations()` combinations("ABC",2) → AB, AC, BC (order not)

ITERTOOLS MUST-KNOW

Function	Behavior	Example/Output
count	infinite 0 1 2...	<code>count(5)</code> → 5,6,7,...
cycle	cycles A B C...	<code>cycle("X","Y")</code> → X,Y,X,Y,...
repeat	repeats 10 10 10	<code>repeat(10,3)</code> → 10,10,10
chain	chains 1 2 3 4	<code>chain([1,2],[3,4])</code> → 1,2,3,4
islice	slices 2 3 4	<code>islice(range(10),2,5)</code> → 1
product	cartesian	<code>2,3,4) ([1,2],("a","b"))</code> →
permutations	order matters AB BA	<code>permutations("AB")</code> BA
combinations	combinations("ABC",2)	→ AB, AC, BC

Key Terms:

- `yield from` — delegates to sub-generator → `yield from sub()` — passes all yields through.
- `send()` — sends value to generator, acts as coroutine → generator receives via `x = yield`
- `throw()` — throws exception inside generator at yield → `gen.throw(ValueError)`.
- `close()` — stops generator immediately → `gen.close()`.
- Coroutine — generator as coroutine, async-like, cooperative multitasking.
- `itertools` — module providing efficient iterators: `count` (infinite), `cycle` (cycles), `repeaters`, `chain` (chains), `islice` (slices) (slices), `permutations`
- `permutations` (combinations (order not)).

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

8 SECTION 8: CONTEXT MANAGERS

★ What is Context Manager?

Definition: Manages resources setup & teardown automatically with `'with'` statement — ensures cleanup even if error occurs. Common example: `'open()'` as `f:` closes automatically. No need manual close!

How `'with'` works? — when you use:

- with `open("file.txt")` as `f:`
- `data = f.read()` # file auto-closed

Without `'with'` (manual way):

```
f = open("file.txt")
try: data = f.read()
    f.close() # you must close manually
```

- `'with'` simplifies this safe, clean, no leaks.
- Guarantees cleanup even if exception raised.

★ How to Create Custom Context Manager?

Class with `__enter__` & `__exit__`:

```
class MyContext:
    • def __enter__(self):
    • print("Enter")
    • return self # what is bound to 'as']
    • def __exit__(self, exc_type, exc_val, exc_tb):
    • print("Exit")
    # handles exception if return True -> suppresses
```

Example usage:

```
with MyContext() as c:
    print("Inside")
```

Output:

Enter -> Inside -> Exit

Using `contextlib` + `@contextmanager` — simpler:

```
from contextlib import contextmanager
@contextmanager
yield "resource" # yielded to 'as' var
print("Exit") # cleanup always runs
with my_context() as r:
    print("Inside") # r = "resource"
Enter -> Inside -> Exit
```

CLASS vs CONTEXTLIB — Comparison

CLASS METHOD	@contextmanager DECORATOR
Define class, implement <code>__enter__</code> & <code>__exit__</code>	Use <code>@contextmanager</code> from <code>contextlib</code>
Exception Handling <code>exc_type, exc_val, exc_tb</code> available; return return <code>True</code> suppresses	Use <code>yield</code> + <code>try/finally</code> <code>yield</code> gives value to <code>'as'</code> ; <code>finally</code> block for cleanup simpler & recommended
Real-World Examples:	
★ with <code>open("file.txt")</code> as <code>f:</code> — file I/O, auto close ★ with <code>threading.Lock()</code> as <code>lock:</code> — acquire/release lock ★ with <code>db.connection()</code> as <code>conn:</code> — DB connection cleanup ★ with <code>timer()</code> as <code>t:</code> — timing context, measure execution ★ Class approach: <code>__enter__</code> returns <code>self</code>; <code>__exit__(exc_type, exc_val, exc_tb)</code>, return <code>True</code> suppresses ★ <code>@contextmanager</code>: uses <code>yield</code> in <code>try/finally</code> — simpler, recommended ★ Real examples: file, lock, DB connection, timer context	

Key Terms: Context Managers —

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

9

SECTION 9: MAGIC METHODS / DUNDER METHODS

★ What is Magic Methods?

Definition: Special methods that start & end with double underscore `__method__` called **dunder** = double underscore. Python calls automatically for operator overloading.

Must-Know Magic Methods:

- `__init__` constructor `__init__(self)` - object creation
- `__str__` string for user `str(obj)` `print(obj)` user friendly
- `__repr__` string for developer `repr(obj)` debugging
- `__len__` `len(obj)` `len`
- `__getitem__` `obj[key]` indexing `obj[0]`
- `__setitem__` `obj[key]=value` setting
- `__del__` destructor
- `__call__` makes object callable `obj()` like function
- `__enter__` / `__exit__` context manager
- `__iter__` / `__next__` iterator
- `__add__` + operator `obj1+obj2`
- `__eq__` == equality
- `__lt__` < less than

Example Class with Magic

```
class MyList:
    • def __init__(self, items):
        self.items = items
    • def __len__(self):
        return len(self.items)
    • def __getitem__(self, index):
        return self.items[index]
    • def __str__(self):
        return f"MyList {self.items}"

ml = MyList([1,2,3])
len(ml)    -> 3
ml[0]      -> 1
print(ml)  -> MyList [1,2,3]
→ note: __len__ __getitem__ __str__ called automatically.
```

MAGIC METHOD	USE	CALLED WHEN
<code>__init__</code>	constructor	obj creation <code>MyClass()</code>
<code>__str__</code>	user string	<code>str()</code> <code>print()</code> <code>print(obj)</code>
<code>__repr__</code>	dev string	<code>repr()</code> debugging
<code>__len__</code>	length	<code>len()</code> <code>len(obj)</code>
<code>__getitem__</code>	indexing [I]	<code>obj[key]</code> <code>obj[0]</code>
<code>__call__</code>	callable ()	<code>obj()</code> like function

What is `__call__`?

Makes object callable

```
class Counter:
    • def __call__(self):
        print("Called")
c () -> Called ← callable object
```

Key Terms: Definitions -

- Magic Methods / dunder: `__method__` starts & ends with double underscore, called automatically, operator overloading
- `__init__` = constructor - creation
- `__str__` = user string - `str()` `print` - user friendly
- `__repr__` = dev string - `repr()` - debugging
- `__len__` = `len` - `len(obj)`
- `__getitem__` = indexing - `obj[key]`
- `__call__` = callable - `obj()` like function
- `__enter__` / `__exit__` = context
- `__iter__` / `__next__` = iterator

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

10) DESCRIPTORS & PROPERTIES (Section 10)

★ What is Descriptor?

Definition box: Object that defines `__get__` `__set__` `__delete__` controls attribute access. Python's way of customizing attribute access used in property.

★ What is @property?

Definition: Built-in descriptor makes method like attribute `getter` `setter` `deleter`.

★ What is Difference property vs descriptor?

property	Descriptor	
is built-in descriptor simpler	@property	@property Descriptor custom class defines <code>__get__</code> <code>__set__</code> <code>__delete__</code> more control reusable

★ Real World Descriptors:

- ORM Django field validation
- Type checking
- Cached property `@cached_property`

★ How Descriptor Works?

```
class Descriptor:
    def __get__(self, obj, objtype=None):
        return value called when attribute accessed
    def __set__(self, obj, value):
        set value called when attribute set
    def __delete__(self, obj):
        delete called when attribute delete
```

Example Usage

```
class MyDesc:
    def __get__(self, obj, objtype=None):
        return 42
    class MyClass:
        attr=MyDesc()
        obj=MyClass()
        obj.attr # 42 __get__ called
```

Example @property getter @setter deleter

```
class Person:
    def __init__(self, name):
        self._name=name
    @property def name(self): return self._name
    @name.setter def name(self, value):
        if not value: raise ValueError("Name cannot be empty")
        self._name=value
    @name.deleter def name(self): del self._name
p=Person("John")
p.name # John getter
p.name= 'Jane' # setter
del p.name # deleter
```

DESCRIPTOR METHODS

Method	Called When	
<code>__get__</code>	called when attribute accessed	getter
<code>__set__</code>	called when attribute set	setter
<code>__delete__</code>	called when attribute deleted del obj.attr	deleter

★ Key Terms box:

- Descriptor `__get__` `__set__` `__delete__` controls attribute access
- customizing attribute access property uses descriptor, `__get__` `getter`
- `obj.attr`, `__set__` `setter` `value`, `__delete__` `deleter` `del obj.attr`,
- `@property` built-in descriptor method like attribute `getter` `setter` `deleter`,
- `@property` `getter` `@name.setter` `setter` `@name.deleter` `deleter`,
- property built-in simple vs Descriptor custom class more control
- reusable ORM validation
- `cached_property`

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

11

SECTION 11 METACLASSES - MOST ADVANCED

★ What is Metaclass?

Class of a class • class that creates classes
type is default metaclass • class is instance of metaclass.

Metaclass = Class of Class =
Creates Classes = type is default metaclass.

★ How Python Classes Created?

class MyClass: pass → Python does
→ type('MyClass', (), {class body})
type creates class → MyClass =
type('MyClass', (), {'x':1})
→ type(name, bases, dict) creates class.

★ What is type?

type is metaclass and type is class of class
type is its own metaclass
type(type) is type.

Example code:

```
type('MyClass', (), {'x':1}) → creates class dynamically
MyClass = type('MyClass', (), {'x':1}) → obj = MyClass]
obj.x → 1
```

★ What is Custom Metaclass?

Class inherits from type → defines __new__
→ controls class creation.

Example code:

```
class MyMeta(type):
    def __new__(cls, name, bases, dict):
        print(f'Creating class {name}')
        dict['new_attr'] = 'added by metaclass'
        return super().__new__(cls, name, bases, dict)
class MyClass(metaclass=MyMeta):
    x = 1
```

CLASS vs METACLASS

Class	Metaclass
• instance of metaclass • creates objects → • __init__ creates instance • MyClass.new_attr • metaclass __new__ called when class defined not when instance created	• instance creates classes • MyClass=type,name,bases,dict) • __new__ creates class • type is default metaclass

★ When to use Metaclass?

Frameworks • ORMs • Django Models • API validation • singleton • registry.

Singleton via Metaclass code:

```
class Singleton(type):
    _instances = {}
    def __call__(cls, *xargs, (*ykarims):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, (*ykarims).
            return cls._instances[cls]
obb1 = MyClass]      obb2 = MyClass]
obb1 is obj2 → True # same instance → singleton
```

★ Key Terms

```
# Metaclass = class of class • creates classes
# type = default metaclass
# class is instance of metaclass,
# type creates class → type(name,bases,dict) →
# type('MyClass', (), {'x':1})
# type is metaclass → type is class of class →
# type is its own metaclass → type(type) is type,
# Custom Metaclass → inherits type → defines __new__
→ controls class creation
# called when class defined not instance,
# Singleton via metaclass → _instances dict → __call__
ensures one instance.
```

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

12

SECTION NAME: 12 GIL & CONCURRENCY - INTERVIEW FAVORITE

★ What is GIL?

Text: Global Interpreter Lock — a mutex that allows only one thread to execute Python bytecode at a time, one thread at a time even on multi-core.

GIL = Global Interpreter Lock = Mutex =
One thread executes bytecode at a time.

★ Why GIL?

Memory management is not thread-safe • GIL simplifies reference counting & garbage collection.

★ What is Impact?

CPU-bound tasks → threads don't help • multi-threaded CPU is slow due to GIL • threads compete for GIL • vs IO-bound tasks help

CPU-BOUND	IO-BOUND	EFFECT OF	SOLUTION/TIP
CPU-bound calc	computation-heavy calc threads don't help • GIL only one thread at a time	• slow	use <code>multiprocessing</code>
IO-bound	waiting releases GIL • threads help threads useful for IO	threads are effective for IO	

★ How to Bypass GIL?

- **Multiprocessing** → uses separate processes • each has its own GIL • own memory space • no GIL limitation
• for CPU-bound use `multiprocessing`
- C extensions → can release the GIL • example: `numpy`
- Alternative interpreters → Jython • IronPython • no GIL

★ What is Multithreading vs Multiprocessing?

THREADING	MULTIPROCESSING
Threading:	• same memory space • shared memory • GIL — one thread at a time
Multiprocessing:	• separate memory space • each process has own GIL • no GIL limit • best for CPU-bound tasks • more memory usage

Example: Threading for IO-bound

```
import threading
def io-task():
    time.sleep(1) # simulate IO wait
threads = [threading.Thread(target=io-task) for _ in range(10)]
# Fast for IO-bound → threads help because IO wait releases the GIL
```

Example: Multiprocessing for CPU-bound

```
import multiprocessing
def cpu-task():
    total = sum(i * i for i in range(10**6)) # heavy computation
processes = [multiprocessing.Process(target=cpu-task) for _ in range(4)]
# Fast for CPU-bound → multiprocessing bypasses GIL via separate processes
```

★ **Key Terms or Interview Tips:** GIL = — mutex • one thread executes bytecode at a time even on multi-core | Why → memory management not thread-safe • reference counting & GC managed by GIL | CPU-bound → threads don't help → use `multiprocessing` for parallelism | IO-bound → threads help • IO waiting releases the GIL → threads are effective | Bypass `multiprocessing`: separate processes each with own GIL • C extensions can release GIL (e.g. `numpy`) • Jython/IronPython have no GIL | Threading → same memory, shared, GIL limited, great for IO-bound, lightweight | Multiprocessing → separate memory, own GIL, no GIL limit, great for CPU-bound, higher memory cost

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

13

SECTION 13: ASYNCIO & CONCURRENCY - MODERN PYTHON

★ What is Asyncio?

Asyncio = Async programming +
Single thread = Event loop +
async/await +
IO-bound.

★ What is async/await?

- async defines corrotine → `async def`
- await waits → pauses corrotine until result ready
- `async def fetch():`
 - `await asyncio.sleep(1)`
 - return data

note: `await` only inside `async def`

How Async Works?

- Event loop runs tasks: when a task awaits IO, the event loop pauses it & runs other tasks instead → no blocking
- Single thread handles many IO tasks cooperatively via cooperative multitasking
- Works great for hundreds of IO operations concurrently, low memory overhead

Example: Concurrent Fetch

```
async def fetch_data(d):
    print(f"Fetching {d}")
    await asyncio.sleep(1) # IO waiting → event loop
    # runs other task
    print(f"Done {d}")
    return data

async def main():
    results = await asyncio.gather(
        fetch_data(1), fetch_data(2), fetch_data(3)
    )

asyncio.run(main())
```

Output: Fetching 1 2 3 → concurrent, not sequential
takes ~1 sec total, not 3 sec → gather runs concurrently

SYNC vs THREADING

vs ASYNCIO

Sync:	sequential, one after another → blocking, slow
Threading:	multiple threads • OS threads • GIL for CPU • IO-bound threads help
Asyncio:	modern favorite single thread • event loop • async/await • cooperative • IO-bound • most efficient for many IO — yellow highlight modern favorite

When to use?

- Sync → simple scripts, easy, CPU-bound small tasks
- Threading → IO-bound, many IO tasks, legacy code, blocking libraries
- Asyncio → IO-bound, many connections, modern, efficient • FastAPI uses asyncio
- Multiprocessing → CPU-bound, heavy compute, bypass GIL

What is FastAPI?

- Modern web framework that uses asyncio • `async def` endpoints → fast, non-blocking

Key Terms ✦

- Asyncio = async programming + single thread + event loop + `async/await` + IO-bound + concurrent
- `async` = defines corrotine → `async def`; `await` = pauses until awaitable finished → only inside `async def`
- Event loop = runs pending tasks, when task awaits IO → switches to other task, single thread handles many IO cooperatively
- `asyncio.gather` = runs multiple coroutines concurrently → concurrent execution → 1 sec for 3 tasks, not 3 sec
- Sync vs Threading vs Asyncio = key difference: blocking vs
- FastAPI = modern web framework → uses asyncio → `async def` endpoints → fast

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

14 SECTION 14: MEMORY MANAGEMENT

★ What is Memory Management?

How Python manages memory allocation & deallocation.

★ What is Reference Counting?

Primary garbage collection – counts references to object; when count 0 → object deleted.

Example:

```
import sys
a = []
sys.getrefcount(a) → counts references
b = a → refcount increases
del b → refcount decreases; when 0 → deleted
```

★ What is Garbage Collector?

Handles reference cycles where objects reference each other; reference counting can't handle cycles. gc module collects cycles.

Example:

```
a = []
b = []
a.append(b)
b.append(a) # cycle: a references b; b references a; refcount never 0 → gc collects.
gc.collect() → manually collects
```

★ What is Memory Leak?

Objects not freed → memory grows.

How to avoid?

- Use weakref → weak references don't increase refcount; import weakref
- Avoid cycles
- Use context managers: with → ensures cleanup

★ What is __slots__?

Saves memory; restricts attributes; no __dict__ Class without slots: each instance has __dict__ → dict overhead; memory heavy. vs with slots: class MyClass: __slots__ = ['x', 'y'] → no __dict__; memory efficient; 40-50% less memory but can't add new attributes.

WITH vs WITHOUT __slots__

Without __slots__	With __slots__
has __dict__ → dict overhead; memory heavy	no __dict__ → memory efficient
can add new attributes; flexible	can't add new attributes; restricted

→ Interview Trap

★ What is Interning?

Small integers -5 to 256 and strings are interned/reused → same object; memory efficient. a=256; b=256 → a is b → True (same object). a=257; b=257 → a is b → False (different). CPython optimization.

Key Terms

- Reference Counting – primary GC; counts references; when 0 → deleted; getrefcount() counts
- Garbage Collector – handles cycles; gc module; gc.collect() manually collects cycle a→b b→a
- Memory Leak – objects not freed → grows; avoid: weakref; cycles; context managers
- __slots__ – Saves memory; no __dict__; 40-50% less; restricted; can't add attributes vs without __dict__: overhead; flexible
- Interning – small ints -5 to 256 & strings reused; same object; memory efficient; is = same object

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

15

SECTION 15 COPY VS SHALLOW & IMMUTABLE MUST KNOW TRAPS

★ What is Shallow Copy?

Copies outer object but inner objects references shared — `copy.copy()` shallow.

Example:

```
import copy
original = [[1,2],[3,4]]
shallow = copy.copy(original)
shallow[0][0] = 99
original → [[99,2],[3,4]] → inner shared, original changed trap!
shallow[0] is original[0] → True (same inner object)
```

★ What is Deep Copy?

Copies outer and inner recursively — `copy.deepcopy()` deep - independent!

Example:

```
deep = copy.deepcopy(original)
deep[0][0] = 99
original unchanged → [[1,2],[3,4]] → deep independent
deep[0] is original[0] → False (different objects)
```

SHALLOW vs DEEP

Aspect	Shallow	Deep
What it does	New outer container, inner shared	New outer & inner, fully cloned
Code	<code>copy.copy(obj)</code>	<code>copy.deepcopy(obj)</code>
Original effect	Original changes if inner mutated	Original unchanged
Identity check	<code>shallow[0] is original[0] → True</code>	<code>deep[0] is original[0] → False</code>
Trap level	High: shared nested objects	Low / safe

favorite trap interview for Deep

Mutable

can change after creation

- list
- set
- bytearray
- mutable trap as default arg

Immutable

cannot change

- int
- float
- str
- tuple
- frozenset
- safe as default

★ What is Mutable Default Argument Trap?

Most famous Python trap!

Code:

```
def func(a, lst=B):
    lst.append(a)
    return lst
func(1) → [1]
func(2) → [1,2] → same list reused!
```

Explain: `lst` default is created once, not fresh each call → trap!

Solution:

```
def func(a, lst=None):
    if lst is None:
        lst = []
    lst.append(a)
    return lst
    use None, not []
```

★ What is Interning and is vs ==?

`==` checks value equality vs `is` checks identity — same object in memory.

Example: 256 is interned → `a = 256, b = 256 → a is b` — True (same object)

• `a = 257, b = 257 → a == b`

True (value same), `a is b` False (different objects) — value same, identity different

★ Key Terms:

- Shallow: `copy.copy()` → outer copied, inner shared references → original changes = trap, `shallow[0]` is `original[0]`
- Deep: `copy.deepcopy()` → outer and inner independent → original unchanged, `deep[0]` is `original[0] = False`
- Mutable: can change → list/dict/set/bytearray → mutable trap with
- Immutable: cannot change → int/float/str/tuple/frozenset →
- Mutable default trap: `def func(a, lst=[])` →
- `lst` created once and reused → trap! Use `None` then if `lst is None: lst = []`
- `is` vs `==`: `==` compares value equality, `is` compares identity (same object in memory) → 256 is interned → `is` True, 257 is not interned → `is` False, `==` is True.

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

16 FUNCTIONAL PROGRAMMING

★ What is Functional Programming?

green box: Programming paradigm treats computation as evaluation of functions avoids changing state immutable.

★ Python Functional Features:

- Lambda: anonymous function `lambda x: x*2`
- `map()` applies function to all items `map(func, iterable)`
`map(lambda x: x*2, [1,2,3])` → `[2,4,6]`
- `filter()` filters based on function `filter(func, iterable)`
`filter(lambda x: x>2, [1,2,3])` → `[3]`
- `reduce()` reduces to single value
`from functools import reduce`
`reduce(lambda x,y: x+y, [1,2,3])` → `6` sum
- `zip()` combines iterables `zip([1,2], ("a","b"))`
→ `[(1,"a"), (2,"b")]`

★ What is Lambda?

Anonymous function
`lambda args: expression x+y`
`lambda x: x*2` `lambda x: else 0`
used with `map` `filter` `sorted`.

Example table:

OBJECT	CODE	RESULT
students. list	<code>students=[("John", "20"),("Jane",22)]</code> <code>sorted(students, key=lambda x:x[1])</code> <code>(("John",20),("Jane",22))</code>	

★ Function Methods Comparison Table:

Function	Def	Example	Result
map	applies func to all	<code>map(lambda x: x*2, [1,2,3])</code>	transforms all → <code>[2,4,6]</code>
filter	filters based on func	<code>filter(lambda x: x>2, [1,2,3])</code>	filters some → <code>[3]</code>
reduce	reduces to single	<code>reduce(lambda x,y: x+y, [1,2,3])</code>	<code>6</code> → single value

★ What is List Comprehension vs Functional?

- List comprehension: Pythonic `[x*2 for x in [1,2,3]]`
→ `[2,4,6]` recommended Pythonic
- vs `map`: `map(lambda x: x*2, [1,2,3])` → `[2,4,6]` functional

green box: List comprehension more Pythonic readable than `map/filter` for simple cases

- Guido prefers.

★ What is functools?

- tools: `reduce` • `lru_cache` • `partial` (fixes args)
- `from functools import partial`
- `def power(a,b): return a**b`
- `square = partial(power, b=2)`
- `square(3)` → `9` power with `b` fixed to `2`

★ Key Terms — Quick Recall

- Functional = computation as evaluation
- avoids changing state
- immutable
- Lambda = anonymous → `lambda args: expression`
- used with `map/filter/sorted`
- `map` = applies to all → `[2,4,6]` transforms
- `filter` = filters → `[3]` keeps some
- `reduce` = reduces to single → `functools.reduce func`
`functools.reduce func iterable` → `6` single
- `zip` = combines → `[(1,"a"), (2,"b")]`
- List comprehension = Pythonic → `[x*2 for x in list]`
- `functools` = `reduce` • `lru_cache` • `partial` → `partial`
- `partial` fixes args → `partial(power, b=2)` →
- `square(3)` → `9`

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

17 SECTION 17: OOP ADVANCED

★ What is OOP Advanced?

Beyond basic classes & inheritance → advanced OOP techniques for scalable, flexible design.

★ What is Inheritance Types?

Single → one parent class. Multiple → multiple parents: `class C(A, B)`.
Multilevel → `A = B → C` chain.
Hierarchical → one parent, multiple children.
Hybrid → combination of above.

★ What is MRO (Method Resolution Order)?

Order Python searches for methods in inheritance. Uses C3 linearization. MRO defines method lookup order. Check via `D.__mro__`.

Example:

```
class A:
    def method(self): print("A")
class B(A):
    def method(self): print("B")
class C(A):
    def method(self): print("C")
class D(B, C): pass
D().method() → B
# MRO: D → B → C → A
```

`D.__mro__` shows the resolution order

★ What is super()?

Calls parent class method. `super().__init__()` calls parent `__init__`:
`super().method()` calls parent method.
Essential for cooperative multiple inheritance.

★ What is Abstract Base Class?

Can't instantiate. Abstract methods must be overridden.
From `abc import ABC, abstractmethod`
from `abc import ABC, abstractmethod`
`class Animal(ABC):`
`def sound(self): pass`
`class Dog(Animal): return "Woof"`
`Animal()` → Error: can't instantiate abstract
`Dog()` → "Woof" must override sound()

★ What is Class vs Instance Variables?

CLASS vs INSTANCE VARIABLE	Instance variable
Shared across all instances <code>MyClass.class_var = 10</code> → one copy shared Access: <code>MyClass.class_var</code>	Unique per instance <code>self.instance_var = 10</code> trap - changing <code>class_var</code> on instance Access: <code>self.instance_var</code>

★ staticmethod vs classmethod vs instance method?

- Instance method → first arg self; needs instance → `obj.method()`
- classmethod → first arg cls; @classmethod; needs class → `MyClass.method()`
- staticmethod → no self/cls; @staticmethod; utility, independent → `MyClass.method()`

Key Terms

- Inheritance → Single, Multiple, Multilevel, Hierarchical, Hybrid
- MRO → Method Resolution Order; C3 linearization; order searches methods; `D.__mro__`
- `super()` → calls parent; `super().__init__()`; cooperative inheritance
- ABC → Abstract Base Class; @abstractmethod; can't instantiate; must override
- Class var → shared; `MyClass.class_var`; one copy vs Instance var → unique; `self.instance_var`; each its own own
- Methods → instance method (self) vs classmethod (@classmethod, cls) vs staticmethod (@staticmethod, no self/cls)

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

18 SECTION 18: EXCEPTION HANDLING & PACKAGING

★ What is Exception Handling Advanced?

Beyond try/except → advanced patterns.
Controls flow when errors occur. Goals:
robustness + cleanup + clearer errors.

try / except / else / finally → flow

try: risky code → attempted operation

- Runs first, may raise exception

except [Exception] as e: handle error

- Runs only if exception raised. e = exception object.

else: runs if NO exception

- Only if try succeeds. Success path.

finally: always runs cleanup

- Runs always, even if exception.

Cleanup like close."

KEY: else = success path | finally = always cleanup
(trap: runs even if error!).

TRY EXCEPT ELSE FINALLY — BEHAVIOR TABLE

Clause	When Runs	Example Use
try	Always start	risky operation (open file)
except	If exception raised	handle error, log, retry
else	If no exception	process result safely
finally	Always, error or not	close file, release resource

Example Code:

```
try:
    f = open("file.txt", "r")
    data = f.read()
except FileNotFoundError:
    + print("Not found") # handle missing
else:
    + print("File read success") # else runs if no err
finally:
    + f.close() # always cleanup, even if error
Output: if file missing → Not found + cleanup runs.
if ok → File read success + cleanup runs.
```

★ What is Custom Exception?

User-defined exception → inherits from Exception class.

class MyError(Exception):

def __init__(self, message):

- super().__init__(message) # init parent
- raise MyError("My custom error") # now raise it
- Use when you need domain-specific errors for clearer code.

★ What is Exception Chaining?

try:

except ZeroDivisionError as e:

raise ValueError("Calculation failed") from e # chains errors

Benefits: keeps cause context → easier debugging.

Trace shows: "During handling of ..., another error occurred: ...".

★ What is Packaging?

How to distribute Python code via pip install.

- Folder with __init__.py — package. Files: module1.py, module2.py.

Structure:

```
my_package/
^-- module1.py      # makes it a package
^-- module2.py      # code here
^-- setup.py         # install config
^-- requirements.txt # deps
^-- README.md
```

setup.py example:

from setuptools import setup

setup(name="my_package",

version="0.1",

packages=["my_package"], install_requires=["requests"]

Now install: pip install -e . for editable install.

★ What is if __name__ == "__main__"?

Guard: runs only if file executed directly, not imported.

if __name__ == "__main__":

main() # only runs when script is run directly

- When imported: __name__ == "my_module" (module name) →

main() NOT run. When run directly:

__name__ == "__main__" → main() RUNS.

Key Terms

- Try/Except/Else/Finally — try risky code; except handle error; else runs if no exception (success); finally always runs cleanup even on exception.

- Custom Exception class MyError(Exception): inherits Exception; raise MyError("msg")

- Exception Chaining raise ... from e → preserves traceback/cause. e.g. __inside__ = "__init__.py" with cause: package; setup.py = name, version, packages, can; can be empty or contain imports to expose.
- if __name__ == "__main__" — runs only if executed directly; is module name when imported vs "__main__" when run directly.

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

19 SECTION 19: TESTING & DEBUGGING & BEST PRACTICES

★ What is Testing?

Ensures code works. Types →

- Unit Testing: tests single function/class → `pytest unittest`
- Integration: tests multiple components together
- E2E (End-to-End): full flow

Example code:

`pytest` modern recommended:

```
def test_add():
    assert add(2,3) == 5
pytest runs test_*.py
```

`unittest` built-in:

```
class TestAdd(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(2,3), 5)
```

★ Fixtures — setup/teardown

```
@pytest.fixture def sample_data():
    • return [1,2,3]
def test_with_fixture/sample_data():
    • assert len(sample_data) == 3
    fixture provides test data
```

★ Mocking — mock object replaces real object

```
from unittest.mock import Mock
mock = Mock()
mock.method.return_value = 5
isolates test — avoids real API/network
```

★ What is Debugging?

Finding & fixing bugs.

- Tools: `pdb` (Python debugger): `import pdb; pdb.set_trace()`
- print debugging: old but works, quick check weakref
- logging: better than print → `import logging`
- logging: better than print → `import logging`
`logging.debug/info/warning/error/critical` → levels
- IDE debugger: PyCharm / VSCode → breakpoints, step through

★ What is Best Practices?

PEP8 = Python style guide → naming conventions
snake_case for functions/variables | PascalCase for classes |
UPPER for constants | 79 chars line

- Type Hints: `def add(a: int, b: int) → int:`
`return a+b`
→ type hints catch errors earlier; better IDE
- Docstrings: `"""Docstring describes function"""` → triple quotes

PYTEST vs UNITTEST

modern recommended	simple assert	class TestCase
<code>def test_add(): assert add()==5</code>		<code>self.assertEqual(...)</code> → verbose/old
fixtures powerful concise		<code>pytest</code> → recommended / modern

DEBUGGING TOOLS

<code>pdb</code>	<code>set_trace</code> / breakpoint → interactive
print	old method → quick but messy
logging	<code>debug/info/warning/error/critical</code> → better than print
IDE debugger	PyCharm / VSCode → breakpoints & visual

Key Terms

- Testing: Unit → single function | `pytest/unittest` | Integration → multiple | E2E - full flow
- `pytest`: modern | simple assert | `test_*.py` | fixtures `@pytest.fixture`
`sample_data` | `mock.Mock` — isolates
- `unittest`: built-in | class `TestCase` | `self.assertEqual` — verbose/old
- Mocking: Mock replaces real → isolates test | `return_value`
- Debugging: `pdb set_trace/breakpoint` → interactive | print → old | logging
`debug/info/warning/error/critical` → IDE debugger
- Best Practices: PEP8 style → snake_case functions | PascalCase classes
UPPER constants | 79 chars
- Type Hints: `def add(a:int,b:int)→int` → catch errors
- Docstrings: `"""..."""` → describes function
- `venv`: virtual env → isolates | `python -m venv env` | source `env/bin/activate`
- `requirements.txt`: pip freeze → `requirements.txt`
- Git: version control

PYTHON ADVANCED HANDBOOK

50 Must-Know Concepts

20 SECTION 20: FINAL RECAP

★ What is Final Recap?

Checklist of 20 Slides Mastered.

- 1 Why Advanced → Beyond basics; senior/Staff FAANG; Execution Model: Code → Bytecode → .pyc → PVM; Zen: Beautiful/Explicit/Simple; Basic vs Advanced: junior vs senior
- 2 Closures → First-Class functions as objects: passed/returned/assigned to var; closure remembers enclosing scope after outer finished; nolocal modifies enclosing scope; counter remembers data; encapsulation
- 3 Decorators → **MOST IMPORTANT!** function takes function + adds/wraps; @syntax wrapper; *args **kwargs handles any; Factory @repeat(n) → returns decorator; Real: @property @staticmethod @classmethod @lru_cache
- 4 Decorators Advanced → functools.wraps preserves metaata: name, docstring; Class Decorator adds to class; Stacking @d1 @d2 → bottom up: d2 first then d1; uses: Timing/logging/caching/auth/validation
- 5 Iterators vs Iterables → Iterable can be iterated via `__iter__` returns iterator (list/tuple/dict for loop); Iterator → `__next__` + `__iter__` maintains state; `next()` → `StopIteration`; `iter()` gets iterator; `next()` gets next; Custom class: `__iter__` returns `__next__`; `__next__` raises `StopIteration`
- 6 Generators → **MOST IMPORTANT!** special iterator using `yield`; lazy, memory efficient; `yield` pauses/saves state/resumes vs return ends; Generator object → `next` → `StopIteration`; Expr: `(x*x for x in range())` → `()` memory efficient vs `[]` list; memory efficient for large data, lazy, infinite; while `True`: `yield` line; one at a time -B/BA: comments AB/AC
- 7 Generator Advanced → `yield` from `self` delegates to sub-generator; `yield` from `sub()`; `send()` send value; `close()`; `throw()`
- 9 Magic Methods (dunder) → `__methom__` start & end with `__`; called automatically; operator overloading: `__init__` constructor; `__str__` user string for print/str user-friendly; `__repr__` dev string for repr/debug; `__len__` `len()`; `__getitem__` `obj[key]`; `__setitem__` `obj[key]=value`; `__call__` → `yield` + `try()` Real: `fileds/threading.Lock` acquire/release; DB conn/timer

Example: repeat decorator

```
def repeat(n):
    def deco(func):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                return func(*args, **kwargs)
        return wrapper
    return deco

@repeat(3) → wraps 3 times
```

★ Top 15 Interview Questions

Q	Question
1	What is decorator?
2	What is closure?
3	Generator vs Iterator?
5	yield vs return?
6	Shallow vs Deep copy?
8	Mutable default trap?
9	Context Manager?
10	Magic Methods?
12	Decorator stacking order?
13	Threading vs Multiprocessing?
14	Asyycio vs Threading?
15	is vs ==?

- 11 Metaclasses → **MOST ADVANCED!** class of class creates classes; type is default metaclass; `type(name,bases,dict) → type("MyClass",(),{"x":1})` dynamically; type is class of class; `type(type)` is type; Custom Metaclass inherits type, defines `__new__` controls class creation when class defined not instance; Singleton via metaclasses: `__instances` dict; `__call__` ensures one instance, same is `True` - shared memory, GIL one thread at a time, good for IO-bound; Multiprocessing: separate memory, own GIL, good for CPU-bound heavy
- 13 Asyncio Concurrency → **MODERN Async**: single-thread event loop, `async/await` for IO-bound concurrent; `async` def defines corotine; `await` waits corotine only inside `async` def; Event loop runs tasks when await IO waiting → runs other `async`; single thread many IO cooperatively; Exmaple: `async def fetch(d): await aiohttp.get(d); aiohttp.gather(...)` runs no `__dict__`; Interning: `smagel ints — 5` `earthname: g. Asyncio`
- 14 Memory Mangemng → Reference Counting: primary GC counts refs; when 0 → deleted; `sys.getrefcount` counts; Garbage Collector handles cycles via gc module `gc.collect()`; `a=[] b=[]`; `a.append(b)`; `b.append(a)` → cycle, refcount never 0 → gc collects; Memory Leak: objects not freed → grows; avoid: weakref, cycles, context managers; `__slots__` saves 40-50% memory no `__dict__`; Interning: small ints -5 to 256 & strings reused same object; is same `step 1+times decing dulle` same object; 256 is True same; 257 is False different
- 10 Copy Deep vs Shallow & Mutalia Trap → Shallow copy `copy()` → ours copied; `new thread executes: objcopy → [2,4,6]`; `filter(func,iter)` filters → `[3]` `reduce(func,iter)` reduces to single via `functools.reduce` → `6`; `__dict__`: 6; `__dict__` combines → `(1,"x"); (2,"b")`; List comp: `ABC: from # do imper ABC, abstratmeter; @lberatmeter; must overrid; Claps var thard vs Insaine var per instance self, ...x own, self.x")` `ovv. fpyth -m, reserivomts..ttk: pip freca → regs.fur`

Save this guide → Share with team!

- Intevuiw Tip: Hands-on practice—build decoators, generators, contuxt managers, write memore-efficient code.